CS 3081 · Artificial Intelligence · Fall 2026

Assignment 1 — Search Strategies

BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

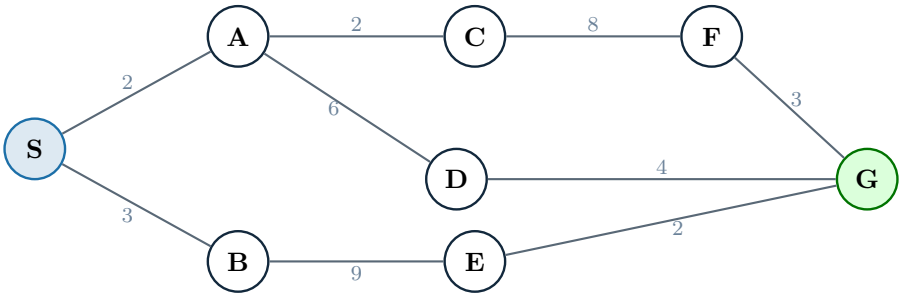
Instructor: Dr Naila Marir **Total marks:** 100
Sections: 3081-1 (CRN 10480), 3081-2 (CRN 10481)

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. S, A, C, G.

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is S and the goal state is G. Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search [8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

Answer: Order of expansion: S, A, B, C, D, E, F, G. Path returned: S, A, D, G.

(b) Depth-first search [8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

Answer: Order of expansion: S, A, C, F, G. Path returned: S, A, C, F, G.

(c) Comparison [6 marks]

BFS and DFS returned different paths. State which path is shorter *in number of steps*, and explain why BFS is guaranteed to find that path while DFS is not.

Answer: BFS's path S, A, D, G is 3 steps; DFS's S, A, C, F, G is 4, so BFS is shorter. BFS expands the graph level by level, so the first time it reaches G it is by a fewest-edges path, which is guaranteed shortest in steps. DFS dives down one branch and returns the first path it finds, A, C, F, G, which need not be shortest.

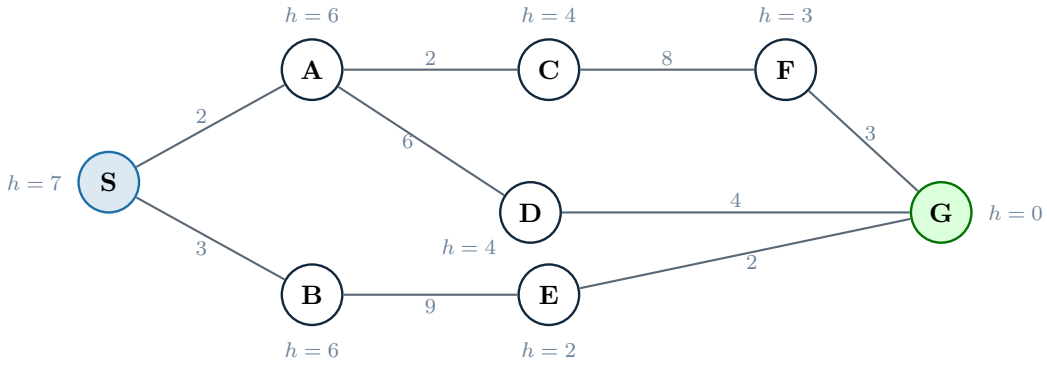
(d) Frontier data structures [3 marks]

BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

Answer: BFS uses a FIFO queue; DFS uses a LIFO stack. The queue removes the oldest node, so BFS expands the shallowest nodes first (level order); the stack removes the newest node, so DFS keeps expanding the most recently discovered node, diving deep before backtracking.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G.



(a) Greedy best-first search [8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Answer: Order of expansion: S, A, C, F, G. Path returned: S, A, C, F, G.

(b) A* search [12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	7	7
2	A	2	6	8
3	C	4	4	8
4	B	3	6	9
5	D	8	4	12
6	G	12	0	12

Order of expansion: S, A, C, B, D, G

Path returned: S, A, D, G **Total cost:** 12

(c) Why they differ [6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Answer: Greedy ignores g(n), the cost already spent, and follows only h, so it chased low heuristics through C then F into the costly C-F (8) and F-G (3) edges (path cost 15). A* uses f = g + h, so it weighed real cost against the estimate, found S, A, D, G cheaper (cost 12), and returned it. Greedy never expanded B or D.

3 Admissibility and optimality

(a) Definition [5 marks]

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

Answer: A heuristic h is admissible if h(n) <= h*(n) for every node n, where h*(n) is the true cost of the cheapest path from n to G (and h(goal) = 0). It never overestimates the remaining cost, i.e. it is optimistic.

(b) Testing the heuristic [10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G, and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	7	6	6	4	4	2	3	0
$h^*(n)$ true	12	10	11	11	4	2	3	0
$h \leq h^*$?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Is h admissible? Yes - no node violates the condition.

If not, name every node that violates the condition: None (h is admissible).

(c) Consequence [6 marks]

Suppose a heuristic overestimates the true cost at exactly one node. Explain what this can do to the solution A* returns, and why BFS could never suffer from this problem.

Answer: If h overestimates at even one node, A* can return a suboptimal (costlier) path: the inflated f-value hides the node on the true cheapest route, so A* reaches G by a worse route and stops there. BFS cannot suffer this because it uses no heuristic at all, expanding by level and ignoring every estimate, so nothing can mislead it.

(d) A trivial heuristic [3 marks]

What does A* reduce to when $h(n) = 0$ for every node? Justify your answer in one sentence.

Answer: It reduces to Uniform-Cost Search (Dijkstra's algorithm): with h(n) = 0, f(n) = g(n), so A* always expands the node with the lowest cost-so-far.

4 Comparing the four strategies

(a) Properties table [8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Optimal?
Breadth-first	FIFO queue	Yes, if all step costs are equal
Depth-first	LIFO stack	No
Greedy best-first	Priority queue on h(n)	No
A*	Priority queue on f = g + h	Yes, if h is admissible

(b) Choosing a strategy [13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

i. A robot must find *any* route out of a building during a fire. Memory is very limited; the route need not be shortest.

Depth-first search - its stack stores only the current path, so memory use stays small (linear in depth), the key limit here; it is not optimal, but the route need not be shortest.

ii. A route planner must return the *cheapest* driving route, and a good straight-line-distance estimate is available.

A* search - with an admissible straight-line heuristic it returns the cheapest route (optimal), and the estimate keeps it far more efficient than uninformed search.

iii. A puzzle has uniform step costs, no useful heuristic, and the solution is known to be shallow.

Breadth-first search - optimal under uniform step costs, needs no heuristic, and the shallow solution keeps its memory use manageable.

(c) Short answer [4 marks]

A* is optimal but is often impractical on large problems. State the reason, and name one property of the search that makes it worse.

Answer: A* must keep every generated node in memory (the whole frontier plus the explored set), so its space complexity is exponential in the worst case; it exhausts memory long before time. The property that makes it worse is memory use, not runtime.

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**